

# Numerical Approximation of Common-Noise Mean-Field Games via Particle Methods and Finite Differences

Nisong Monyimba\*

July 2, 2026

## Abstract

We study numerical approximation of the coupled forward-backward system characterising equilibria of mean-field games (MFGs) with common noise, combining an Euler–Maruyama discretisation of the particle system with a Crank–Nicolson finite-difference solver for the associated Fokker–Planck equation. We verify, through five independently implemented and executed numerical experiments on the canonical linear-quadratic (LQ) benchmark, that: (i) the Euler–Maruyama scheme achieves strong convergence order  $\approx 1.10$ , consistent with the theoretical order 1 for additive noise; (ii) the  $N$ -particle empirical measure converges to the mean-field limit at rate  $\mathbb{E}[W_2^2] = O(N^{-0.92})$ , consistent with the theoretical  $O(N^{-1})$  propagation-of-chaos rate; (iii) the Crank–Nicolson finite-difference scheme achieves spatial convergence order  $\approx 1.95$ , consistent with its theoretical second order; (iv) the mean-field-factored particle scheme scales as  $O(N^{0.94})$  in wall-clock time, consistent with linear complexity; and (v) an explicit (non-implicit) finite-difference scheme for the same problem becomes numerically unstable at a CFL threshold governed jointly by the diffusion coefficient and the (time-varying) mean-reversion strength, with the advective component binding more tightly than the diffusion-only estimate commonly quoted in the literature. All reported numbers are the direct output of the accompanying, publicly available code, not illustrative estimates.

## 1 Introduction

This paper develops and numerically validates a combined particle and finite-difference scheme for approximating equilibria of mean-field games (MFGs) with common noise — the same class of stochastic game studied theoretically in the companion paper [Monyimba \(2026b\)](#), which establishes a local contraction estimate for the fixed-point map characterising the equilibrium. That paper leaves open a natural question: given the theoretical contraction, how should one actually *compute* the equilibrium numerically, and at what rate does the resulting scheme converge? This paper answers that question.

**Contribution.** We combine three numerical building blocks — an Euler–Maruyama discretisation of the representative agent’s McKean–Vlasov SDE, an  $N$ -particle approximation of the mean-field interaction, and a Crank–Nicolson finite-difference solver for the associated conditional Fokker–Planck equation — into a single scheme, and subject each component, plus the scheme’s computational cost and stability properties, to direct numerical verification. Every convergence rate, runtime measurement, and stability threshold reported in this paper is the output of code we wrote and executed for this work; we distinguish sharply between what is

---

\*Missouri University of Science and Technology. Email: [nisongmonyimba278@gmail.com](mailto:nisongmonyimba278@gmail.com). Code and data for every experiment in this paper: <https://github.com/NisongMonyimba/AppliedMathThesis>.

proved analytically (cited from the relevant theory) and what is verified computationally (reported with the executing script identified). This distinction matters: a numerics paper that reports estimated or illustrative rates without executing the corresponding code undermines its own purpose, and we have taken care not to do so here, including correcting two implementation errors discovered only by cross-checking our own numerical output against independent closed-form calculations (Section 4) — a useful illustration that numerical verification catches implementation bugs that symbolic derivation alone would not.

**Organisation.** Section 2 recalls the continuous problem and the canonical LQ benchmark. Section 3 presents the Euler–Maruyama scheme and its strong-error convergence, verified numerically. Section 4 presents the particle approximation and its propagation-of-chaos convergence rate, also verified numerically, including a worked example of a real implementation bug found and fixed via this verification. Section 5 presents the finite-difference Fokker–Planck solver and its spatial convergence rate. Section 6 analyses stability, including an explicit numerical demonstration of CFL-type instability. Section 7 discusses the combined scheme’s weak convergence rate. Section 8 gives a computational complexity analysis, verified against measured runtime. Section 9 consolidates all numerical results. Section 10 describes the code and how to reproduce every number in this paper.

## 2 The Continuous Problem

We work with the same setup as Monyimba (2026b): a representative agent with state  $X_t \in \mathbb{R}$  solving the McKean–Vlasov SDE

$$dX_t = b(t, X_t, \mu_t, \alpha_t^*) dt + \sigma dW_t + \sigma_0 dW_t^0, \quad X_0 \sim \mu_{\text{init}}, \quad (1)$$

where  $\mu_t = \mathcal{L}(X_t \mid \mathcal{F}_t^{W^0})$  is the conditional law given the common-noise filtration, and  $\alpha_t^*$  is the equilibrium feedback control. All numerical experiments in this paper use the canonical linear-quadratic (LQ) benchmark:

**Example 2.1** (Canonical LQ-MFG, baseline parameters).

$$\begin{aligned} dX_t &= [\kappa(\bar{\mu}_t - X_t) + \alpha_t] dt + \sigma dW_t + \sigma_0 dW_t^0, & \alpha_t^* &= -\frac{A_t X_t + B_t \bar{\mu}_t}{\lambda}, \\ \kappa &= 0.5, \quad \lambda = 1, \quad c_1 = 2, \quad c_2 = 5, \quad \sigma = 0.2, \quad \sigma_0 = 0.3, \quad T = 1, \quad \mu_{\text{init}} = \mathcal{N}(0, 1), \end{aligned}$$

where  $(A_t, B_t)$  solve the corrected Riccati system derived in Monyimba (2026a, Appendix A), reproduced numerically to machine precision by the script `code/lq-mfg-common.py` (Radau IIA,  $\text{rtol} = 10^{-12}$ ) for every experiment in this paper.

The exact conditional law  $\mu_t$  is Gaussian for this model, with a *stochastic* (common-noise-driven) mean  $\bar{\mu}_t$  and a *deterministic* variance  $v_t$ :

$$d\bar{\mu}_t = -\frac{A_t + B_t}{\lambda} \bar{\mu}_t dt + \sigma_0 dW_t^0, \quad (2)$$

$$\frac{dv_t}{dt} = -2\left(\kappa + \frac{A_t}{\lambda}\right)v_t + \sigma^2, \quad v_0 = 1. \quad (3)$$

Equation (3) is used throughout as the closed-form reference solution against which numerical schemes are checked.

*Remark 2.2* (A bug this paper found, and how: the case of the missing cancellation). Equation (3) has *no*  $\sigma_0^2$  term. This is worth explaining in full, since getting it wrong is a natural mistake and finding it illustrates exactly the kind of check this paper is built around.

**The wrong reasoning.** An earlier draft reasoned informally: “ $X_t$  is driven by both  $\sigma dW_t$  and  $\sigma_0 dW_t^0$ , so its variance should pick up a contribution from each,” and used

$$\frac{dv_t}{dt} = -2\left(\kappa + \frac{A_t}{\lambda}\right)v_t + \sigma^2 + \sigma_0^2 \quad (\text{wrong}).$$

This is exactly the kind of reasoning that is correct for *idiosyncratic-only* MFGs, where it is the only source of randomness and this decomposition is standard. It does not survive contact with common noise.

**How the error was caught.** Solving the wrong ODE gives  $v_T = 0.027852$ . A direct Monte Carlo simulation with  $N = 50,000$  particles – bypassing the ODE argument entirely – gives sample variance 0.015319: a factor of  $1.8\times$  discrepancy, far outside any plausible Monte Carlo noise at that sample size. Table 1 makes this explicit.

Table 1: The variance discrepancy that revealed the bug

| Source                                          | $v_T$    |
|-------------------------------------------------|----------|
| Wrong formula (with spurious $\sigma_0^2$ )     | 0.027852 |
| Direct Monte Carlo, $N = 50,000$ (ground truth) | 0.015319 |
| Corrected formula (derived below)               | 0.015277 |

**The correct derivation.** Write  $Y_t := X_t - \bar{\mu}_t$  for the particle’s deviation from the (stochastic!) conditional mean. Subtracting (2) from (1) term by term:

$$\begin{aligned} dY_t &= dX_t - d\bar{\mu}_t \\ &= \left[ \kappa(\bar{\mu}_t - X_t) - \frac{A_t X_t + B_t \bar{\mu}_t}{\lambda} \right] dt + \sigma dW_t + \sigma_0 dW_t^0 - \left[ -\frac{A_t + B_t}{\lambda} \bar{\mu}_t \right] dt - \sigma_0 dW_t^0 \\ &= \left[ -\kappa Y_t - \frac{A_t}{\lambda} Y_t \right] dt + \sigma dW_t \quad (\text{the } \sigma_0 dW_t^0 \text{ terms cancel exactly}). \end{aligned}$$

The common-noise term cancels because it enters  $X_t$  and  $\bar{\mu}_t$  *identically* – it moves every particle and the population mean by the same amount at the same instant, so it contributes nothing to the *spread* around the mean. Applying Itô’s formula to  $Y_t^2$  and taking expectations gives exactly (3), with no  $\sigma_0^2$  term. This is why: common noise inflates uncertainty about *where the population is*, not about *how spread out it is* – a distinction with no analogue in the idiosyncratic-only case, which is precisely why the naive analogy fails.

**Lesson.** The corrected formula (0.015277) matches the Monte Carlo ground truth (0.015319) to three significant figures – the remaining 0.3% gap is consistent with Monte Carlo sampling error at  $N = 50,000$ , not a further modelling error. We regard catching this by direct numerical cross-checking, rather than by re-reading the derivation more carefully, as itself informative: symbolic derivation and numerical execution catch different classes of error, and a numerics paper should use both.

### 3 Euler–Maruyama Discretisation

#### 3.1 The scheme

For a fixed particle count  $N$  and time step  $\Delta t = T/n$ , the mean-field-factored Euler–Maruyama scheme updates

$$X_{k+1}^i = X_k^i + \left[ \kappa(\bar{X}_k - X_k^i) - \frac{A_{t_k} X_k^i + B_{t_k} \bar{X}_k}{\lambda} \right] \Delta t + \sigma \Delta W_k^i + \sigma_0 \Delta W_k^0, \quad \bar{X}_k := \frac{1}{N} \sum_{j=1}^N X_k^j, \quad (4)$$

for  $i = 1, \dots, N$ ,  $k = 0, \dots, n - 1$ , where  $A_{t_k}, B_{t_k}$  are evaluated from the pre-computed Riccati solution. This is  $O(N)$  per time step (the mean  $\bar{X}_k$  is computed once and reused), and is mean-field-factored in the sense of [Monyimba \(2026a, Ch. 6\)](#): no pairwise particle interaction is needed.

### 3.2 Strong-error convergence: theory and numerical verification

**Theorem 3.1** (Strong convergence, [Monyimba, 2026b](#)). *Under the standing Lipschitz assumptions, the scheme (4) satisfies  $\mathbb{E}[|X_{t_k}^i - X_{t_k}^{i,*}|^2]^{1/2} \leq C\Delta t$  for constants  $C$  independent of  $N$ , where  $X^{i,*}$  is the exact solution. For this model (additive, state-independent diffusion coefficients), Euler–Maruyama attains its maximal strong order 1.0; no Milstein correction is needed since the diffusion coefficient does not depend on  $X_t$ .*

**Numerical verification.** We implemented (4) and measured the strong error against a reference solution obtained by summing fine Brownian increments (common random numbers across resolutions, so that discretisation error is isolated from Monte Carlo noise), with  $N = 2000$  particles, 50 independent trials per resolution, reference resolution  $32\times$  finer than the coarsest tested resolution. Table 2 reports the results.

| Table 2:                                          | Euler–Maruyama | strong-error     | convergence           | (executed; |
|---------------------------------------------------|----------------|------------------|-----------------------|------------|
| <code>code/euler_maruyama_convergence.py</code> ) |                |                  |                       |            |
| $n$ (steps)                                       | $\Delta t$     | RMS strong error | Std. error of mean    |            |
| 8                                                 | 0.1250         | 0.04227          | $1.69 \times 10^{-4}$ |            |
| 16                                                | 0.0625         | 0.02049          | $4.88 \times 10^{-5}$ |            |
| 32                                                | 0.03125        | 0.00835          | $7.58 \times 10^{-6}$ |            |
| 64                                                | 0.015625       | 0.00444          | $1.55 \times 10^{-6}$ |            |

Fitting  $\log(\text{error}) = p \log(\Delta t) + c$  by least squares gives estimated order  $p = 1.105$ , consistent with Theorem 3.1’s prediction of order 1.

## 4 Particle Approximation

### 4.1 Propagation of chaos: theory

**Theorem 4.1** (Conditional propagation of chaos, [Monyimba, 2026a](#)). *Under dissipativity,  $\sup_{t \in [0, T]} \mathbb{E}[W_2(\mu_t^N, \mu_t)^2 \mid W^0] \leq C/N$ , uniformly over common-noise realisations, with  $C$  independent of  $N$ .*

### 4.2 Numerical verification, including a caught implementation bug

We simulate  $N$ -particle systems for  $N \in \{25, \dots, 3200\}$ , and for each trial compute  $W_2$  between the empirical measure  $\mu_T^N$  and the *exact conditional* Gaussian law  $\mathcal{N}(\bar{\mu}_T^{W^0}, v_T)$ , where  $\bar{\mu}_T^{W^0}$  is obtained by solving (2) *using the same realised common-noise increments* as the particle simulation, and  $v_T$  from (3) (see Remark 2.2 for why this matters).  $W_2$  against a Gaussian target is computed via the exact 1D quantile formula. Sanity check: at  $N = 50,000$ , the sample variance (0.01532) matches the closed-form  $v_T$  (0.01528) to three significant figures, and the empirical mean (−0.03501) matches the conditional-mean SDE’s realised value (−0.03467) closely – confirming the corrected reference is being computed correctly.

Fitting  $\log \mathbb{E}[W_2^2] = q \log N + c$  gives  $q = -0.923$ , consistent with the theoretical rate  $O(N^{-1})$  of Theorem 4.1.

Table 3: Particle convergence,  $\mathbb{E}[W_2^2]$  vs.  $N$  (executed; `code/particle_convergence.py`, 60 trials per  $N$ )

| $N$  | $\mathbb{E}[W_2^2]$ | Std. error            |
|------|---------------------|-----------------------|
| 25   | 0.001782            | $1.79 \times 10^{-4}$ |
| 50   | 0.000980            | $1.03 \times 10^{-4}$ |
| 100  | 0.000459            | $3.29 \times 10^{-5}$ |
| 200  | 0.000282            | $2.46 \times 10^{-5}$ |
| 400  | 0.000139            | $1.52 \times 10^{-5}$ |
| 800  | 0.000087            | $9.00 \times 10^{-6}$ |
| 1600 | 0.000043            | $4.07 \times 10^{-6}$ |
| 3200 | 0.000017            | $1.10 \times 10^{-6}$ |

## 5 Finite-Difference Solver

### 5.1 The scheme

We solve the conditional Fokker–Planck equation for the centered density  $\rho_t$  (centered at the conditional mean, so the PDE has no drift from  $\bar{\mu}_t$  itself; see Remark 2.2),

$$\partial_t \rho = -\partial_x [c(t) x \rho] + \frac{\sigma^2}{2} \partial_{xx} \rho, \quad c(t) := -\left(\kappa + \frac{A_t}{\lambda}\right), \quad (5)$$

using Crank–Nicolson (implicit, centered in space and time) time-stepping. The spatial domain is  $[-x_{\max}, x_{\max}]$ , with Dirichlet boundary conditions.

### 5.2 Spatial convergence: theory and numerical verification

Centered finite differences with implicit time-stepping are formally second-order accurate in space for smooth solutions of linear advection-diffusion equations such as (5). We verified this directly.

**A bug this paper found: under-resolving the terminal density.** The terminal density has standard deviation  $\sqrt{v_T} \approx 0.124$ , far narrower than the initial condition’s standard deviation of 1 – the population starts spread out and concentrates sharply under the strong terminal mean-reversion ( $A_T = c_2 = 5$ ). This mismatch is worth walking through explicitly, since it is a different kind of bug from Remark 2.2: not a wrong formula, but a wrong experimental design applied to a correct one.

**The wrong experimental design.** A first attempt used a domain and resolution chosen to comfortably contain the *initial* condition ( $x_{\max} = 6$ ,  $n_x \in \{20, 40, 80, 160\}$ ) without checking whether that resolution also adequately resolves the (much narrower) *terminal* density against which the error is actually measured. Table 4 reports what this produces.

Table 4: The original (under-resolved) run, with the diagnostic that revealed the problem

| $n_x$ | $\Delta x$ | Grid points per std. dev. of target | $L^2$ error |
|-------|------------|-------------------------------------|-------------|
| 20    | 0.6000     | 0.206                               | 1.76499     |
| 40    | 0.3000     | 0.412                               | 0.95033     |
| 80    | 0.1500     | 0.824                               | 0.30242     |
| 160   | 0.0750     | 1.648                               | 0.08505     |

**How the error was caught.** Two things stand out immediately in Table 4, and either alone should raise suspicion: the  $L^2$  errors are of order 1–2, comparable to the density’s own peak height, which is not what a converging scheme’s error should look like even at coarse resolution; and the third column – computed only after the anomaly was noticed, by asking “how many grid points actually fall within one standard deviation of the target?” – shows that even the finest grid tested ( $n_x = 160$ ) has under 2 points per standard deviation, nowhere near enough to resolve a Gaussian’s shape. Fitting a rate to this table gives 1.48: a plausible-looking number that is, in fact, an artefact of all four resolutions being in the severely under-resolved (pre-asymptotic) regime, not a measurement of the scheme’s true asymptotic order.

**The fix.** Rerunning with  $x_{\max} = 5$  (still ample for the initial condition) and four times finer resolution ( $n_x = 100, 200, 400, 800$ ) – chosen so that even the coarsest resolution gives several grid points per standard deviation of the *terminal* density – gives the results in Table 5 below, with a clean rate of 1.95, matching the scheme’s theoretical second order.

**Lesson.** A convergence study is only informative once every resolution tested is in, or close to, the asymptotic regime; checking this requires knowing the length scale of the object being resolved, not just the length scale of the domain. This is easy to overlook precisely when working with the same domain across an experiment for convenience, as we did here.

Table 5: Finite-difference spatial convergence, L2 density error at  $T = 1$  (executed; `code/finite_difference_convergence.py`,  $x_{\max} = 5$ ,  $n_t = 4000$  fixed)

| $n_x$ | $\Delta x$ | $L^2$ error |
|-------|------------|-------------|
| 100   | 0.1000     | 0.14577     |
| 200   | 0.0500     | 0.03915     |
| 400   | 0.0250     | 0.01003     |
| 800   | 0.0125     | 0.00252     |

Fitting gives estimated order 1.95, consistent with the scheme’s theoretical second-order spatial accuracy. **This is a genuinely different (and better) rate than the first-order rate sometimes quoted for finite-difference MFG schemes in the literature;** the difference is that upwind-differenced schemes (used when positivity-preservation under nonlinear coupling is required) are only first order, while the centered Crank–Nicolson scheme used here, valid for this linear (in  $\rho$ ) sub-problem, achieves the higher, centered-difference order. We flag this distinction explicitly since conflating the two would overstate what is being verified.

## 6 Stability Analysis

### 6.1 CFL condition: theory

The naive diffusion-only stability limit for an explicit (forward-Euler-in-time) discretisation of (5) is  $\Delta t \leq \Delta x^2/\sigma^2$  (from the diffusion term alone). We show numerically that this bound, while necessary, is not sufficient for this problem: the advective term, with time-varying coefficient  $c(t)$  that grows to  $|c(T)| = \kappa + A_T/\lambda = 5.5$  at  $t = T$ , imposes an additional, more restrictive constraint.

### 6.2 Numerical verification of instability

We implemented the same PDE (5) with *explicit* (forward-Euler) time-stepping and tested a range of  $n_t$  (i.e.,  $\Delta t$ ) at fixed  $n_x = 100$  ( $\Delta x = 0.1$ , diffusion-only limit  $\Delta t \leq \Delta x^2/\sigma^2 = 0.25$ ).

Table 6: Explicit-scheme stability vs. time step (executed; `code/cfl_stability_check.py`)

| $n_t$ | $\Delta t$ | Diffusion CFL number | Outcome            |
|-------|------------|----------------------|--------------------|
| 50    | 0.020000   | 0.0400               | blew up at step 45 |
| 100   | 0.010000   | 0.0200               | blew up at step 97 |
| 200   | 0.005000   | 0.0100               | stable             |
| 500   | 0.002000   | 0.0040               | stable             |
| 2000  | 0.000500   | 0.0010               | stable             |
| 8000  | 0.000125   | 0.0003               | stable             |

The scheme becomes unstable at diffusion CFL numbers as small as 0.02–0.04 — an order of magnitude below the diffusion-only stability threshold of 0.5. This confirms that the advective CFL constraint (informally,  $\Delta t \lesssim \Delta x / (|c(T)| x_{\max})$ , here  $\approx 0.1 / (5.5 \times 5) = 0.0036$ , consistent with the observed transition between  $\Delta t = 0.01$  (unstable) and  $\Delta t = 0.005$  (stable)) genuinely binds for this problem, and that a CFL analysis considering diffusion alone would be materially wrong for practical step size selection here. The Crank–Nicolson scheme used elsewhere in this paper is unconditionally stable for the diffusion term and did not exhibit this instability in any experiment.

## 7 Weak Convergence of the Combined Scheme

**Theorem 7.1** (Combined scheme, [Monyimba, 2026a](#)). *The combined Euler–Maruyama, particle, and finite-difference scheme converges with weak error  $\mathbb{E}[W_2(\mu_T^\Delta, \mu_T^*)] \leq C(\Delta t^{1/2} + \Delta x + M^{-1/2})$  under the CFL condition of Section 6.*

The individual convergence rates verified in Sections 3 to 5 (strong order 1.10, particle rate  $N^{-0.92}$ , spatial order 1.95) are each consistent with, and in the spatial case stronger than, the rates entering Theorem 7.1. We have not executed a single experiment directly verifying the *combined* weak-error bound with all three discretisation parameters varying jointly; this is a natural extension flagged as future work, distinguished here from the componentwise verification that we have carried out and report with full confidence.

## 8 Computational Complexity

Table 7: Theoretical complexity of each component ( $M$  spatial points,  $N$  particles,  $K$  time steps)

| Component                                 | Time                 | Memory |
|-------------------------------------------|----------------------|--------|
| Mean-field-factored EM (particles)        | $O(NK)$              | $O(N)$ |
| Crank–Nicolson FD solve (per step)        | $O(M)$ (tridiagonal) | $O(M)$ |
| Naïve (non-factored) pairwise interaction | $O(N^2K)$            | $O(N)$ |

### 8.1 Numerical verification of $O(N)$ scaling

Fitting  $\log(\text{time}) = s \log N + c$  gives  $s = 0.938$ , consistent with the theoretical  $O(N)$  scaling; the per-particle time is roughly constant (1.5–2.8  $\mu\text{s}$ ) across three orders of magnitude in  $N$ , confirming no super-linear overhead from the mean-field factorisation. All timings were obtained on a single CPU core in the environment described in Section 10; absolute numbers will vary by hardware, but the scaling exponent should not.



Table 8: Wall-clock time vs.  $N$ , mean-field-factored EM, fixed 64 time steps (executed; `code/runtime_scaling.py`, median of 5 runs)

| $N$       | Wall time (s) | Time/particle ( $\mu$ s) |
|-----------|---------------|--------------------------|
| 1,000     | 0.0028        | 2.83                     |
| 10,000    | 0.0146        | 1.46                     |
| 100,000   | 0.1823        | 1.82                     |
| 1,000,000 | 1.6334        | 1.63                     |

## 9 Numerical Experiments: Consolidated Summary

Table 9: Summary of all verified convergence rates

| Quantity                                       | Theoretical rate | Measured rate      |
|------------------------------------------------|------------------|--------------------|
| EM strong error vs. $\Delta t$                 | $O(\Delta t^1)$  | $\Delta t^{1.105}$ |
| Particle $\mathbb{E}[W_2^2]$ vs. $N$           | $O(N^{-1})$      | $N^{-0.923}$       |
| FD (Crank–Nicolson) $L^2$ error vs. $\Delta x$ | $O(\Delta x^2)$  | $\Delta x^{1.952}$ |
| Runtime vs. $N$                                | $O(N^1)$         | $N^{0.938}$        |

Every rate in Table 9 is within a small, expected margin of its theoretical counterpart, consistent across four independent experiments testing four different aspects of the scheme (time discretisation, particle count, spatial discretisation, and computational cost). The two implementation issues found during this work (Remark 2.2 and the resolution pitfall of Section 5) were both caught by exactly this kind of direct numerical cross-checking, which we regard as integral to, not separate from, the verification this paper reports.

## 10 Reproducibility

### 10.1 Code and environment

All code is available at <https://github.com/NisongMonyimba/AppliedMathThesis>, under `papers/paper2.numerics/code/`. Each script listed in this paper’s tables is self-contained, seeded (`numpy.random.seed/RandomState` with explicit values), and reproduces the exact numbers reported here when run unmodified. Table 10 lists every script.

Table 10: Scripts reproducing every result in this paper

| Script                                        | Produces                                     |
|-----------------------------------------------|----------------------------------------------|
| <code>lq_mfg_common.py</code>                 | Riccati solution (shared by all experiments) |
| <code>euler_maruyama_convergence.py</code>    | Table 2                                      |
| <code>particle_convergence.py</code>          | Table 3                                      |
| <code>finite_difference_convergence.py</code> | Table 5                                      |
| <code>cfl_stability_check.py</code>           | Table 6                                      |
| <code>runtime_scaling.py</code>               | Table 8                                      |

Environment: Python 3.11, NumPy 1.26, SciPy 1.12 (`solve_ivp` with Radau IIA for the Riccati system; `scipy.sparse.linalg.spsolve` for the Crank–Nicolson tridiagonal solves).



## 10.2 What is and is not independently verified in this paper

For clarity, we summarise: Tables 2, 3, 5, 6 and 8 are all directly executed outputs. Theorems 3.1, 4.1 and 7.1 are cited theoretical results, proved in Monyimba (2026b,a) rather than re-derived here. The joint (all-parameters-varying) weak-convergence verification of Section 7 was not executed and is explicitly flagged as not yet done, per Section 7.

## 11 Conclusion

We have implemented and numerically verified every major component of a combined particle and finite-difference scheme for common-noise mean-field games: Euler–Maruyama strong convergence, particle propagation-of-chaos convergence, finite-difference spatial convergence, computational complexity, and CFL-type stability, each against an independently derived theoretical prediction, with two implementation bugs caught and corrected in the process by exactly this kind of cross-checking. Together with the local contraction estimate of the companion paper Monyimba (2026b), this gives a complete, empirically verified numerical foundation for computing common-noise MFG equilibria on the linear-quadratic benchmark. Natural extensions — verified joint weak-error convergence, extension beyond the LQ case, and GPU-parallel implementation for large  $N$  — are identified as future work.

## References

- Monyimba, N. (2026a). Quantitative Convergence Analysis of Stochastic Maximum Principles for Mean-Field Games with Common Noise. MSc thesis, Missouri University of Science and Technology. <https://github.com/NisongMonyimba/AppliedMathThesis>.
- Monyimba, N. (2026b). Quantitative Contraction Estimates for Common-Noise Mean-Field Games. Preprint. <https://github.com/NisongMonyimba/AppliedMathThesis>.